

Linux Memory Management

A comprehensive, undergraduate-friendly guide to how Linux manages memory: virtual addresses and page tables, page faults, the buddy and slab allocators, the page cache, reclaim and swap, and the OOM killer, from `malloc()` down to physical page frames.

Generated for CMSC 125 ICS UPLB

Reviewed by JACH

Includes 15 diagrams, annotated kernel walk-throughs, six hands-on labs with real output, worked page-replacement examples, and review questions with answers.

Contents

Before You Begin	5
The whole story in one paragraph	5
Prerequisites	5
Roadmap	5
Why Memory Management?	7
1.1 The problem	7
1.2 What the kernel must do	7
1.3 The memory hierarchy	7
1.4 Physical vs virtual: the two views of memory	7
Virtual Memory and Paging	9
2.1 Pages and page frames	9
2.2 Address translation	9
2.3 Why page tables are multi-level	10
2.4 The page table entry	11
2.5 The TLB: making translation fast	11
2.6 Huge pages	12
2.7 Kernel and user in one address space	12
The Process Address Space in the Kernel	13
3.1 mm_struct and VMAs	13
3.2 Seeing VMAs: /proc/PID/maps	13
3.3 Creating and changing mappings	14
3.4 The four kinds of mapping	14
3.5 Heap and stack	14
3.6 ASLR	15
Page Faults and Demand Paging	16
4.1 Lazy allocation: memory is a promise	16
4.2 What is a page fault?	16
4.3 Inside the fault handler	17
4.4 Copy-on-write	17
4.5 Counting faults	18
Physical Memory Management	19
5.1 Tracking page frames	19
5.2 Nodes and zones	19

5.3 The buddy allocator	19
5.4 Watermarks: how much free memory to keep	20
5.5 Allocation flags	21
Kernel Memory Allocation	22
6.1 The slab allocator (SLUB)	22
6.2 kmalloc vs vmalloc	23
6.3 The direct map	23
6.4 Kernel memory is not free either	23
malloc() in User Space	24
7.1 Two ways to get memory from the kernel	24
7.2 Inside the allocator	24
7.3 Typical memory bugs	24
The Page Cache and File I/O	26
8.1 Using free RAM as a disk cache	26
8.2 Reads, writes, and writeback	26
8.3 Memory-mapped files	26
8.4 Reading free	27
Reclaim, Swap, and Page Replacement	28
9.1 When memory runs low	28
9.2 What can be reclaimed?	28
9.3 Choosing victims: the LRU lists	28
9.4 Swap	29
9.5 Textbook page-replacement algorithms	30
Overcommit and the OOM Killer	31
10.1 Allocating more than you have	31
10.2 The three overcommit modes	31
10.3 When the promise is called in: the OOM killer	31
Advanced Topics	33
11.1 Huge pages	33
11.2 NUMA	33
11.3 Pinning memory and shared memory	33
11.4 Memory cgroups	34
11.5 Memory protection features	34
Observing Memory	35
12.1 The tools	35

12.2 Key fields of /proc/meminfo	35
12.3 Per-process sizes: VSZ, RSS, PSS, USS	36

Hands-On Labs 37

Lab 1: Demand paging, one fault per page	37
Lab 2: Overcommit, the promise vs the reality	38
Lab 3: Copy-on-write after fork, measured with PSS	38
Lab 4: malloc, brk, and mmap	39
Lab 5: The two kinds of segmentation fault	40
Lab 6: Watching the page cache	41

Review, Misconceptions, and Reference 42

14.1 Common misconceptions	42
14.2 Where to find it in the source	42
14.3 Review questions	43
14.4 Answer key	44
14.5 Glossary	45
14.6 References and further reading	46

START HERE

Before You Begin

Every program you run believes it has a large, private, uninterrupted block of memory all to itself. That belief is an illusion, and maintaining it is one of the most important jobs of an operating system. Behind the scenes, the Linux kernel is juggling a limited supply of physical RAM among hundreds of processes, loading data only when it is needed, sharing what can be shared, moving cold data out to disk, and protecting every process from every other.

This guide explains how that works on Linux, from a call to `malloc()` in your C program to the physical page frames that back it. We keep asking two questions: *what does the program think it has?* and *what does the kernel actually do?*

The whole story in one paragraph

Programs use **virtual addresses**. The CPU's **MMU** translates them to **physical addresses** in 4 KiB units called **pages**, using **page tables** that the kernel maintains for each process. A process's valid address ranges are recorded as **VMAs**. Memory is handed out lazily: `malloc` and `mmap` merely *promise* address space, and the real page is supplied on the first touch, through a **page fault (demand paging)**. Physical pages come from the **buddy allocator**; small kernel objects come from the **slab allocator**. Files are cached in the **page cache**, which uses otherwise idle RAM. When free memory runs low, the kernel **reclaims** pages: it drops clean cache, writes dirty pages back, and pushes cold anonymous pages to **swap**. If that is not enough, the **OOM killer** terminates a process.

Prerequisites

- Basic C: pointers, arrays, dynamic allocation with `malloc`.
- OS lecture basics: process, address space, system call, user mode vs kernel mode.
- A Linux shell. The labs use `gcc`, `strace`, `free`, and `/proc`.

Scope. This guide is self-contained. It briefly explains anything it needs from other topics (such as processes and `fork`) at the point where it is used.

Roadmap

Chapter	Question it answers
1 Why memory management?	What problems is the kernel solving? What does the memory hierarchy look like?
2 Virtual memory and paging	How does a virtual address become a physical one? What are page tables and the TLB?
3 The process address space	What are VMAs? What do <code>mmap</code> , <code>brk</code> , and <code>mprotect</code> do?
4 Page faults and demand paging	Why is memory allocated lazily? What are minor and major faults?
5 Physical memory	What are zones, page frames, and the buddy allocator? What are watermarks?
6 Kernel memory allocation	How does the kernel allocate small objects? <code>kmalloc</code> , <code>slab</code> , <code>vmalloc</code>
7 malloc in user space	What does <code>malloc</code> really do? When is memory returned?

Chapter	Question it answers
8 The page cache	Why does a file read get faster the second time? What is writeback?
9 Reclaim, swap, replacement	What happens when RAM runs low? FIFO, LRU, Clock, and what Linux does
10 Overcommit and the OOM killer	Why can you allocate more than you have? Who gets killed, and why?
11 Advanced topics	Huge pages, NUMA, mlock, shared memory, cgroups, protections
12 Observing memory	How to read <code>free</code> , <code>/proc/meminfo</code> , RSS, PSS, <code>smaps</code>
13 Hands-on labs	Six experiments with real output
14 Review	Misconceptions, source map, questions, answers, glossary, references

NOTE — ABOUT THE EXAMPLES

Sample outputs were produced on Ubuntu 24.04 (glibc 2.39) with a Linux 6.x kernel in a small virtual machine (about 4 GB of RAM, one CPU, no swap). Numbers on your machine will differ, but the *patterns* will match. Kernel code shown is **simplified** for learning; real functions have many extra checks. Newer kernels use "folios" (groups of pages) in many code paths that older text describes as pages.

CHAPTER 1

Why Memory Management?

1.1 The problem

Imagine a computer with 4 GB of RAM running a browser, an editor, a compiler, and a music player at once. Without memory management, each program would have to know which parts of RAM the others use, and any bug in one could overwrite another (or the OS itself). Programs could not be bigger than RAM, and RAM could not be shared or rearranged. Memory management solves these problems.

1.2 What the kernel must do

- **Abstraction.** Give every process the illusion of its own large, contiguous address space.
- **Isolation and protection.** Prevent a process from reading or writing memory it does not own, and separate user code from kernel code.
- **Allocation.** Track which physical pages are free and hand them out quickly, with little waste.
- **Sharing.** Let processes share memory safely when useful: one copy of the C library, shared memory segments, and copy-on-write after `fork`.
- **Efficiency.** Use RAM as a cache for disk, keep hot data in memory, and evict cold data.
- **Overload handling.** Decide what to do when memory is scarce: reclaim, swap, or kill.

1.3 The memory hierarchy

Fast memory is small and expensive; large memory is slow and cheap. Memory management is largely about keeping the right data in the right level. The figures below are rough orders of magnitude for typical hardware; the exact numbers vary widely.

Level	Typical size	Typical access time	Managed by
CPU registers	under 1 KB	under 1 ns	Compiler
L1 / L2 / L3 cache	32 KB to tens of MB	1 to 40 ns	Hardware
Main memory (RAM)	GBs	about 50 to 100 ns	The kernel (this guide)
SSD	hundreds of GB to TBs	about 50 to 100 microseconds	Kernel (swap, page cache, file system)
Hard disk	TBs	about 5 to 10 ms	Kernel (swap, page cache, file system)

A disk access is roughly **a hundred thousand to a hundred million times slower** than a register or cache access. That gap is why the kernel works hard to avoid disk I/O, and why running out of RAM makes a system slow rather than merely a little slower.

1.4 Physical vs virtual: the two views of memory

- **Physical memory** is the actual RAM, divided into fixed-size **page frames** (4 KiB on x86-64). Only the kernel deals with physical addresses.

- **Virtual memory** is the view each process has: a range of addresses (up to 128 TiB in the user half on x86-64) that need not correspond to anything in RAM at a given moment. Each page can be in RAM, on disk (swap or a file), or not yet created.

KEY IDEA

A virtual address means nothing without the process's page tables. The same number, say `0x7f0000000000`, can refer to completely different physical memory in two different processes, or to no memory at all.

CHAPTER 2

Virtual Memory and Paging

2.1 Pages and page frames

Virtual memory is divided into fixed-size **pages**; physical memory into **page frames** of the same size. On x86-64 (and ARM64 usually) the page size is **4096 bytes (4 KiB)**. You can check it: `getconf PAGESIZE` printed **4096** on the test machine. Paging solves several classic problems at once:

- **No external fragmentation** for user memory: any free frame can hold any page, so a process's memory need not be contiguous in RAM.
- **Isolation**: each process has its own mapping from pages to frames.
- **Sharing**: two processes can map the same frame (for shared libraries, for example).
- **Bigger than RAM**: pages that are not in RAM can live on disk until needed.

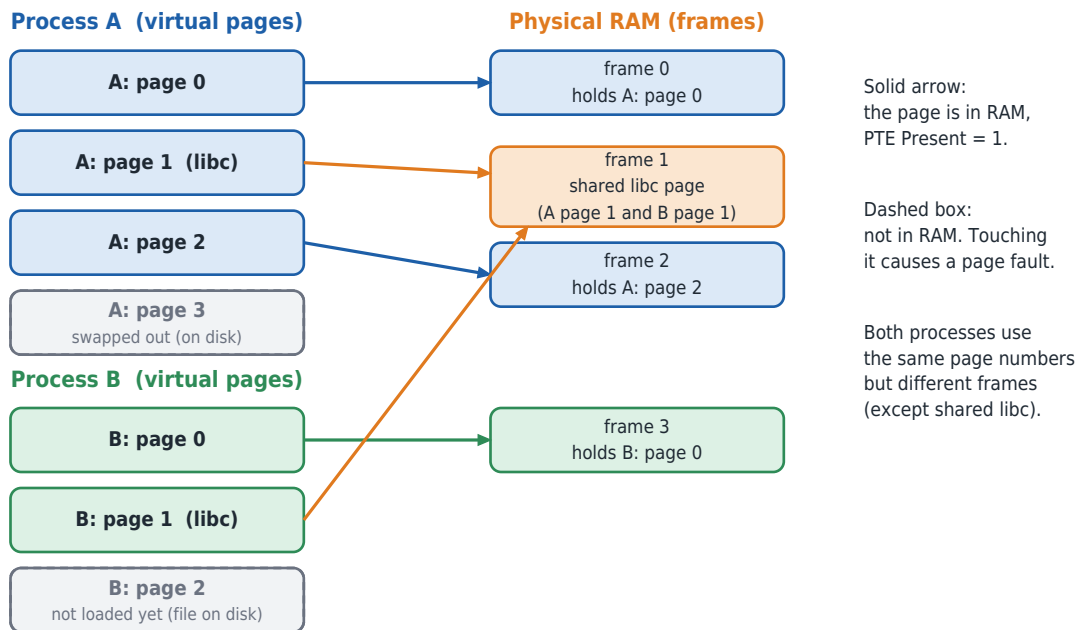


Figure 2.1: Two processes with their own virtual pages. Page tables map them to physical frames, sometimes shared, sometimes not in RAM at all.

2.2 Address translation

A 48-bit virtual address is split in two parts: the **virtual page number** (the high bits) and the **offset within the page** (the low 12 bits, since $4096 = 2^{12}$). Translation replaces the page number with a **physical frame number** and keeps the offset unchanged.

```
virtual address = [ virtual page number | offset (12 bits) ]
                  |           |
                  |           |
          page table lookup   |
                  v           |
physical address = [ physical frame number | offset (unchanged) ]
```

Example. Take the real address `0x00007f57d7a01234` from a test run. The offset is `0x234` (the low 12 bits) and the virtual page number is `0x7f57d7a01`. The page table lookup finds which frame holds that page, say frame `F`. The physical address is then $F * 4096 + 0x234$.

2.3 Why page tables are multi-level

A single flat table with one entry per virtual page would be enormous. A 48-bit address space has 2^{48} pages, and at 8 bytes per entry that is **512 GiB of page table per process**: absurd, and almost all of it would describe pages nobody uses. So the tables are a **tree**. On x86-64 there are four levels; each level is a 4 KiB table of 512 eight-byte entries, indexed by 9 bits of the address ($9 \times 4 + 12 = 48$ bits). Lower-level tables are allocated only for address ranges that are actually in use.

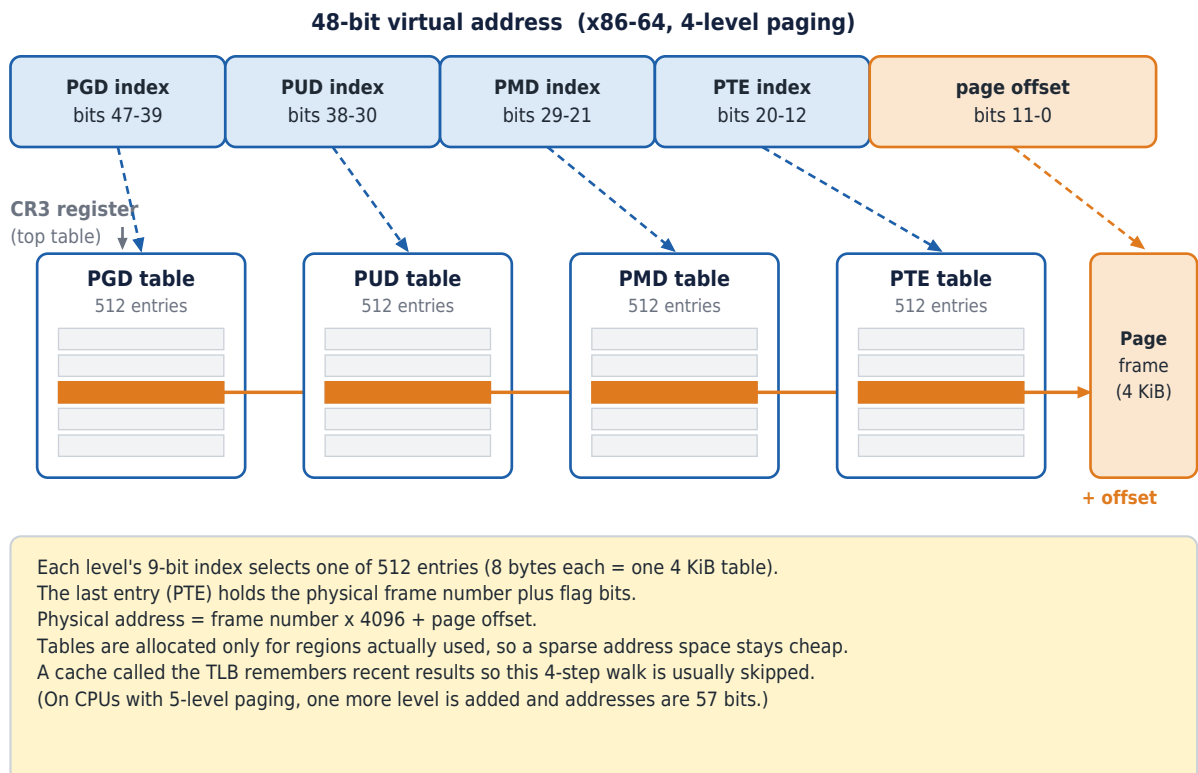


Figure 2.2: A four-level page-table walk on x86-64.

Linux name	Address bits	Entry points to
PGD (page global directory)	47 to 39	a PUD table
PUD (page upper directory)	38 to 30	a PMD table (or a 1 GiB huge page)
PMD (page middle directory)	29 to 21	a PTE table (or a 2 MiB huge page)
PTE (page table entry) level	20 to 12	a 4 KiB page frame
page offset	11 to 0	byte within the page

For the example address `0x00007f57d7a01234`, the four 9-bit indexes are 254, 351, 189, and 1, and the offset is `0x234`. The CPU register **CR3** holds the physical address of the top (PGD) table of the running process. Each process has its own PGD, and switching processes means loading a different value into CR3.

NOTE — 48 BITS OR 57 BITS?

The test machine's `/proc/cpuinfo` reported "57 bits virtual". That CPU supports **5-level paging**, which adds one more level (called P4D) and extends addresses to 57 bits. The 4-level, 48-bit layout shown here is still the most common. The idea is the same at every depth.

2.4 The page table entry

A page table entry (PTE) on x86-64: 64 bits

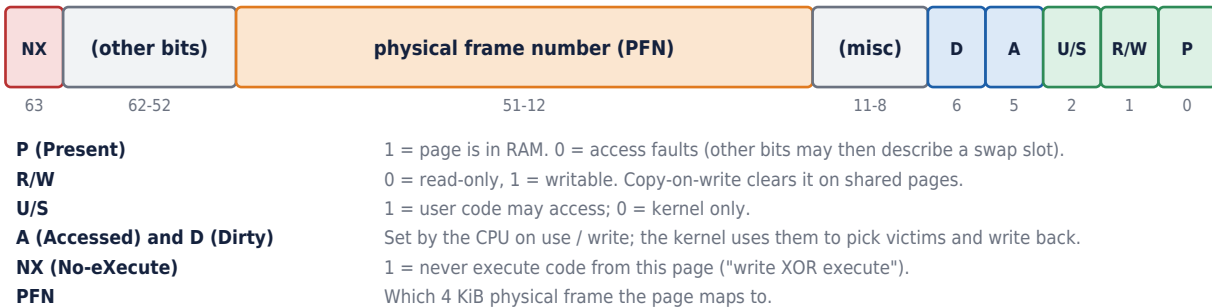
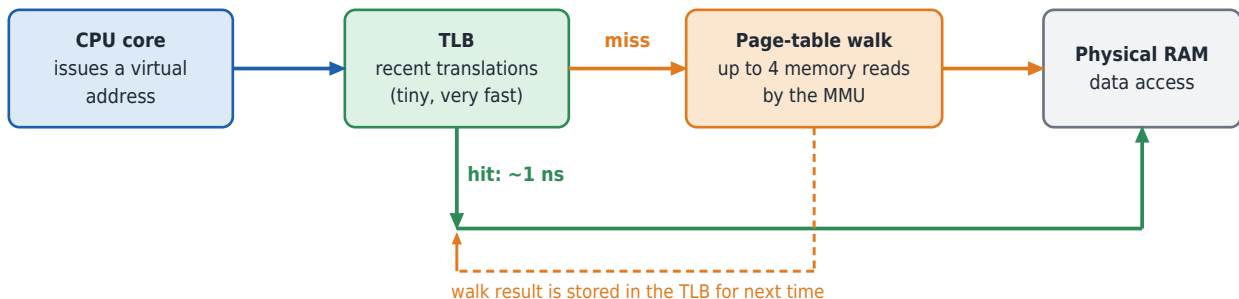


Figure 2.3: The bits of an x86-64 page table entry that matter most.

The entry does more than translate. Its flag bits carry **permissions** (read/write, user/kernel, no-execute) and **usage information** (accessed, dirty) that the kernel relies on. When the **Present** bit is 0, the hardware ignores the other bits and raises a page fault; the kernel is free to use those bits to remember where the page really is (a swap slot, for example).

2.5 The TLB: making translation fast

A translation that needed four extra memory reads for every memory access would slow the machine to a crawl. The CPU therefore keeps a small, very fast cache of recent translations called the **TLB** (translation lookaside buffer).



The TLB is flushed (or tagged) when the CPU switches to a different address space.

Figure 2.4: The TLB sits between the CPU and the page-table walk.

- On a **hit**, translation costs about as much as a cache access. On a **miss**, hardware walks the page tables (up to four memory reads) and refills the TLB.

- When the kernel changes a mapping (unmapping a page, changing permissions, or writing protecting pages for copy-on-write), stale TLB entries must be invalidated. On multicore machines this may require interrupting other CPUs, called a **TLB shutdown**, which is one reason changing mappings is not free.
- On a context switch to a different process, the old translations are wrong. CPUs avoid a full flush by tagging entries with an address-space ID (PCID on x86).

2.6 Huge pages

With 4 KiB pages, a program using 1 GiB needs 262,144 page table entries (2 MiB of PTEs) and, crucially, many TLB entries. **Huge pages** map a much larger unit with one entry: **2 MiB** (at the PMD level) or **1 GiB** (at the PUD level). One TLB entry then covers 512 times more memory. Chapter 11 covers how Linux uses them.

2.7 Kernel and user in one address space

On x86-64, the address space is split in halves. The lower half (128 TiB) belongs to the process; the upper half is the kernel, mapped into every process's page tables but **marked supervisor-only** in the U/S bit. That is why a system call does not need to switch page tables: the kernel is already mapped. A user program touching kernel addresses gets a page fault and is killed with **SIGSEGV**. (Since the Meltdown vulnerability, many systems use **KPTI**, which unmaps most kernel memory while user code runs, at some cost in speed.)

CHAPTER 3

The Process Address Space in the Kernel

3.1 mm_struct and VMAs

The kernel describes each process's address space with a `struct mm_struct`. It contains the pointer to the top-level page table and a collection of `vm_area_struct` objects, called **VMAs**. Each VMA describes one **contiguous range of virtual addresses** with the same properties: permissions, whether it is private or shared, and what backs it (a file and offset, or nothing). The collection is a maple tree in recent kernels (older ones used a red-black tree and a linked list).

VMAs (`vm_area_struct`), one per contiguous region

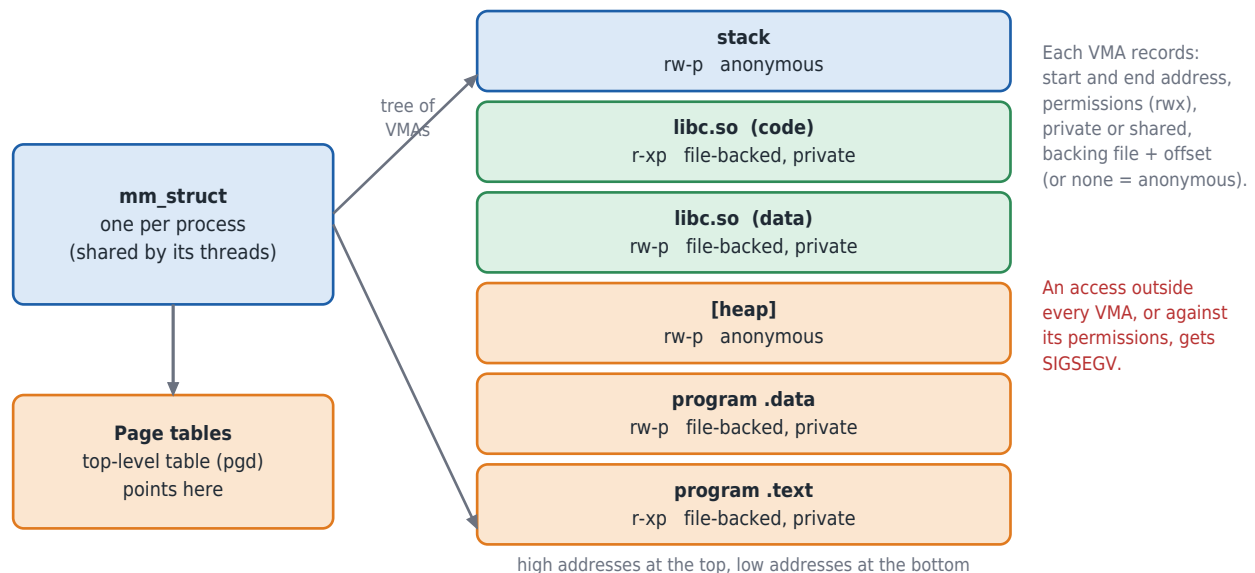


Figure 3.1: An `mm_struct` with its VMAs and page tables. The VMAs say what is legal; the page tables say what is currently in RAM.

KEY IDEA

VMAs and page tables are different things. A VMA is the kernel's *promise*: this range is valid and this is what it means. Page table entries are the *current state*: this page is (or is not yet) in RAM. A page can be inside a VMA and have no PTE at all yet. That is exactly the situation demand paging exploits (Chapter 4).

3.2 Seeing VMAs: `/proc/PID/maps`

Each line of `/proc/<PID>/maps` is one VMA. A trimmed listing from a real process:

address range	perms	backing	
55ba429fc000-55ba429fd000	r-xp	/home/claude/plabs/p6	program code
55ba429ff000-55ba42a00000	rw-p	/home/claude/plabs/p6	program data + bss
55ba743dd000-55ba743fe000	rw-p	[heap]	brk() heap
7fec07228000-7fec073b0000	r-xp	/usr/lib/x86_64-linux-gnu/libc.so.6	libc code
7fec07403000-7fec07405000	rw-p	/usr/lib/x86_64-linux-gnu/libc.so.6	libc data
7fec075f2000-7fec075f4000	r-xp	[vdso]	kernel-provided code
7ffee11cd000-7ffee11ee000	rw-p	[stack]	the stack

The permission letters are **r**ead, **w**rite, **e**xecute, and **p**riate (copy-on-write) or **s**hared. Code is never writable and data is never executable; the CPU enforces this through the PTE's R/W and NX bits.

3.3 Creating and changing mappings

System call	What it does
<code>mmap(addr, len, prot, flags, fd, off)</code>	Create a new VMA: map a file, or anonymous zero-filled memory. The workhorse of memory management.
<code>munmap(addr, len)</code>	Remove mappings; frees the pages and PTEs.
<code>mprotect(addr, len, prot)</code>	Change permissions (for example make a region read-only or executable).
<code>brk(addr) / sbrk(n)</code>	Move the end of the heap (the "program break"), growing or shrinking the heap VMA.
<code>madvise(addr, len, advice)</code>	Hint how a region will be used (<code>MADV_DONTNEED</code> , <code>MADV_HUGEPAGE</code> , ...).
<code>mremap, mlock, msync</code>	Move or resize a mapping; pin pages in RAM; flush a mapping to its file.

3.4 The four kinds of mapping

`mmap` flags give a 2 x 2 matrix. Almost everything in a process falls into one of these four boxes:

	Private (MAP_PRIVATE)	Shared (MAP_SHARED)
File-backed	Program code and data, shared libraries. Reads see the file. Writes make a private copy (copy-on-write) and never reach the file.	Memory-mapped file I/O. Writes go to the shared page cache and to the file; other processes see them.
Anonymous (MAP_ANONYMOUS)	Heap, stack, big <code>malloc</code> blocks. Zero-filled on first touch. Private to the process (and copied to a child after <code>fork</code> , lazily).	Shared memory between related processes (created before <code>fork</code>), and <code>shm/tmpfs</code> memory.

3.5 Heap and stack

- **Heap.** A VMA that grows upward as `brk()` raises the program break. glibc's `malloc` uses it for small requests (Chapter 7).
- **Stack.** A VMA at the top of user space that **grows downward automatically**: touching just below the stack's current end makes the kernel extend the VMA, up to the limit `ulimit -s` (typically 8 MB). A guard gap separates the stack from other mappings so that an overflow becomes a fault rather than silent corruption.
- **mmap region.** Shared libraries and large allocations live between them. The dynamic linker `ld.so` maps libraries here when the program starts.

3.6 ASLR

The kernel randomizes the starting addresses of the stack, heap, mmap region, and (for position-independent programs) the executable itself on every run, so attackers cannot predict where code and data are. Run any program that prints a stack address twice and the values differ. Real output from two runs of one test program: `0x7fff0d98869c` on the first run and `0x7ffc90c916cc` on the second. The control file is `/proc/sys/kernel/randomize_va_space` (2 = full).

CHAPTER 4

Page Faults and Demand Paging

4.1 Lazy allocation: memory is a promise

When you call `malloc(1<<30)` or `mmap` a large region, the kernel does **not** find a gigabyte of RAM. It just creates a VMA. Physical pages are allocated on the **first access**, one page at a time. This is **demand paging**, and it is why programs start fast (only code that runs is loaded), why big sparse arrays are cheap, and why `VmSize` and `VmRSS` in `/proc` are very different numbers.

4.2 What is a page fault?

A **page fault** is an exception the CPU raises when a memory access cannot proceed: the PTE is missing, its Present bit is 0, or the access is not permitted (a write to a read-only page, for example). Control passes to the kernel's fault handler, which decides what to do. The word "fault" is misleading: most faults are **normal** and are the mechanism by which the system works.

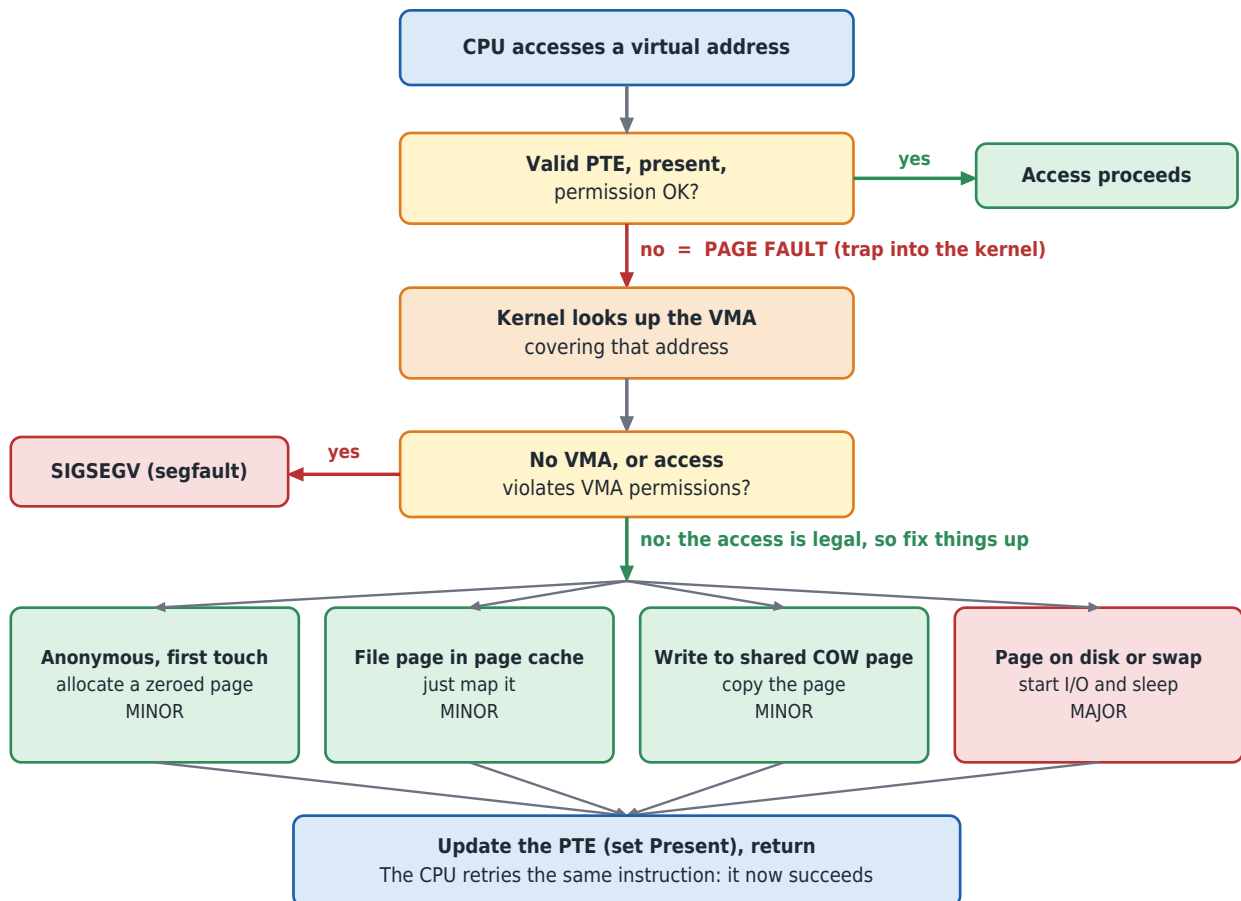


Figure 4.1: What the kernel does on a page fault.

Kind	Cause	Cost	Outcome
Minor fault	Page can be supplied without disk I/O: first touch of anonymous memory (zero page), file page already in the page cache, or copy-on-write copy	Microseconds or less	Page mapped; program continues
Major fault	Page must be read from disk: file data not cached, or a page that was swapped out	Milliseconds (disk) or tens of microseconds (SSD)	Process sleeps during I/O, then continues
Invalid fault	No VMA covers the address, or the access violates the VMA's permissions	n/a	SIGSEGV (or SIGBUS for a file mapping beyond the end of the file)

4.3 Inside the fault handler

```

page fault exception (CPU)
-> do_user_addr_fault()           // arch/x86/mm/fault.c
    vma = find_vma(mm, address)    // is this address inside a VMA?
    if (!vma or access not permitted) // no -> SIGSEGV
        bad_area()
    handle_mm_fault(vma, address, flags) // mm/memory.c
    -> handle_pte_fault()
        if pte is empty:
            anonymous VMA -> do_anonymous_page() // allocate a zeroed page, map it
            file VMA -> do_fault() // page cache lookup, then map
        else if pte not present:
            do_swap_page() // read the page back from swap
        else if write to a write-protected page:
            do_wp_page() // copy-on-write: copy, map writable
    return; the CPU re-executes the faulting instruction

```

4.4 Copy-on-write

Copy-on-write (COW) uses the same machinery. When a process calls `fork()`, the kernel creates a child with its own page tables but does **not copy the data**. Both processes' PTEs point to the same frames, and every writable private page is marked read-only in both. The first write by either process causes a fault; the kernel then copies that one page for the writer.

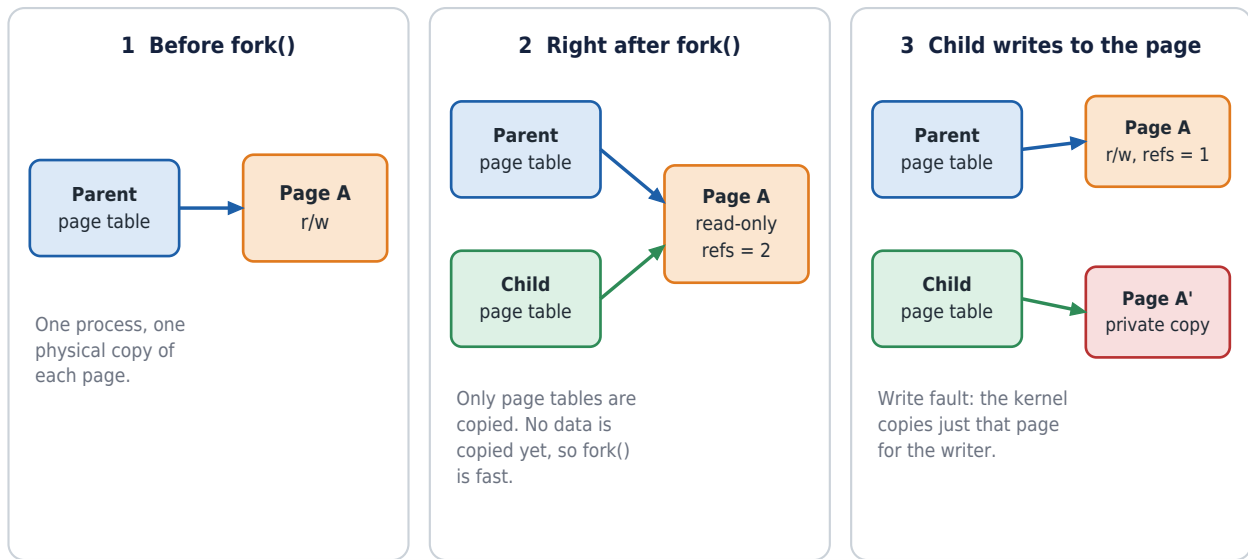


Figure 4.2: Copy-on-write after fork(). Only the page tables are copied at fork time.

- Because most children call `exec` right after `fork`, most pages are never copied at all, and `fork` stays cheap.
- COW also makes it safe to share one physical copy of the zero-filled page: reading untouched anonymous memory can map a shared zero page, and only the first write allocates a real one.

4.5 Counting faults

Fault counters are easy to read: `ps -o pid,min_flt,maj_flt,cmd, /proc/<PID>/stat, getrusage()`, and the global `pgfault` and `pgmajfault` counters in `/proc/vmstat`. Lab 1 shows demand paging happening live, one fault per page.

WATCH OUT

A high **major** fault rate is a red flag: the process is waiting for disk, and the machine is probably short of RAM or reading cold data. A high **minor** fault rate is often harmless (a program starting up, or touching a lot of newly allocated memory).

CHAPTER 5

Physical Memory Management

5.1 Tracking page frames

Every physical page frame has a small descriptor in the kernel, a `struct page` (or, increasingly, a `struct folio` covering a group of pages). It records how the frame is used, a reference count, flags (dirty, locked, on the LRU, ...), and which mapping owns it. A descriptor is typically 64 bytes, so the kernel spends about **1.6%** of RAM (64 / 4096) just to keep track of the rest: around 64 MiB on a 4 GiB machine.

5.2 Nodes and zones

The kernel divides physical memory into **nodes** (one per NUMA memory bank; ordinary machines have just one) and each node into **zones**, because not all memory is equally usable for every purpose.

Physical address space of an x86-64 machine (not to scale)



DMA: old devices that can only address 24 bits.

DMA32: devices limited to 32-bit addresses.

NORMAL: everything else, used for most allocations.

(ZONE_MOVABLE may also exist for memory that can always be migrated, e.g. for hot-unplug.)

Figure 5.1: Memory zones on x86-64.

5.3 The buddy allocator

The kernel needs to hand out physical pages quickly, sometimes in **contiguous groups** (for DMA buffers or huge pages), without fragmenting memory into unusable crumbs. Linux uses the **buddy system**. Free memory is kept in lists by **order**: a block of order k has 2^k contiguous pages (order 0 = 4 KiB, order 1 = 8 KiB, ... order 10 = 4 MiB).

1 Start: one free block of 16 pages (order 4)**2 Allocate 2 pages (order 1): split 16, then 8, then 4; hand out one 2-page block****3 Free those 2 pages: their buddy is free, so they merge ...****4 ... 2+2 -> 4, 4+4 -> 8, 8+8 -> 16: back to one big block**

Figure 5.2: Splitting a block to satisfy a request and merging buddies when pages are freed.

- **Allocate order k:** take a block from the order-k free list. If empty, take a bigger block and **split** it in half repeatedly, keeping one half and putting the other on the lower list.
- **Free:** if the freed block's **buddy** (the equal-sized neighbor) is also free, **merge** them into a block of the next order, and repeat. This keeps large blocks available. Two blocks are buddies if they are the same size, adjacent, and their addresses differ in exactly one bit.
- **Per-CPU page lists** cache single pages for each CPU so most order-0 allocations need no lock.
- **Fragmentation:** if memory is full of scattered single pages, large contiguous allocations fail even when plenty of memory is free. The kernel groups pages by mobility (movable, unmovable, reclaimable) and can run **compaction** to move pages together.

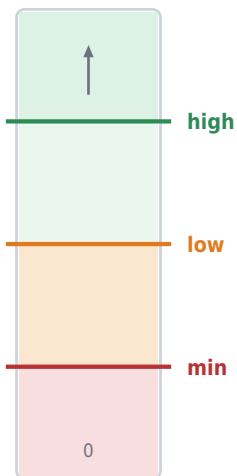
You can see the free lists in `/proc/buddyinfo`. Each column is the count of free blocks of orders 0, 1, 2, ... 10. Real output:

```
$ cat /proc/buddyinfo
Node 0, zone DMA      0      0      0      0      0      0      0      0      0      1      1      3
Node 0, zone DMA32    1      1      1      1      1      2      2      2      1      2    750
Node 0, zone Normal   1      3      0      2      3      1      2      1      2      1      8
                        order 0                                order 10
```

The DMA32 zone has 750 free blocks of order 10 (4 MiB each), about 3000 MiB in unfragmented pieces. This machine was freshly started, so memory is barely fragmented.

5.4 Watermarks: how much free memory to keep

The kernel does not wait until memory is completely exhausted. Each zone has three **watermarks**: **min**, **low**, and **high**, in pages. They are visible in `/proc/zoneinfo` (for the Normal zone on the test machine: min 686, low 857 pages).

free pages in a zone**Plenty free.**

kswapd sleeps.

Below high, above low.

Still fine; a running kswapd stops at high.

Below LOW: kswapd wakes up

and reclaims pages in the background.

Below MIN: allocating tasks do

DIRECT RECLAIM themselves (they stall).

If reclaim cannot free enough, the OOM killer is the last resort (Chapter 10).

Figure 5.3: Zone watermarks and the reclaim actions they trigger.

The tunable `vm.min_free_kbytes` (67584 KB on the test machine) sets the overall reserve, from which the per-zone watermarks are derived. The reserve exists so that the kernel itself can still allocate memory (for example while writing pages out to free memory) even in emergencies.

5.5 Allocation flags

Kernel code passes **GFP flags** to say what an allocation may do. `GFP_KERNEL` means "you may sleep and start reclaim to satisfy this"; `GFP_ATOMIC` means "you may not sleep" (interrupt handlers), so it can dip into reserves and fail more easily. Flags also choose the zone and whether the memory is movable.

CHAPTER 6

Kernel Memory Allocation

The buddy allocator hands out whole pages, but the kernel constantly needs small objects: a 5888-byte `task_struct`, a 192-byte `dentry`, a 64-byte buffer. Allocating a full 4 KiB page for each would waste enormous amounts of memory. So the kernel builds smaller allocators on top of the page allocator.

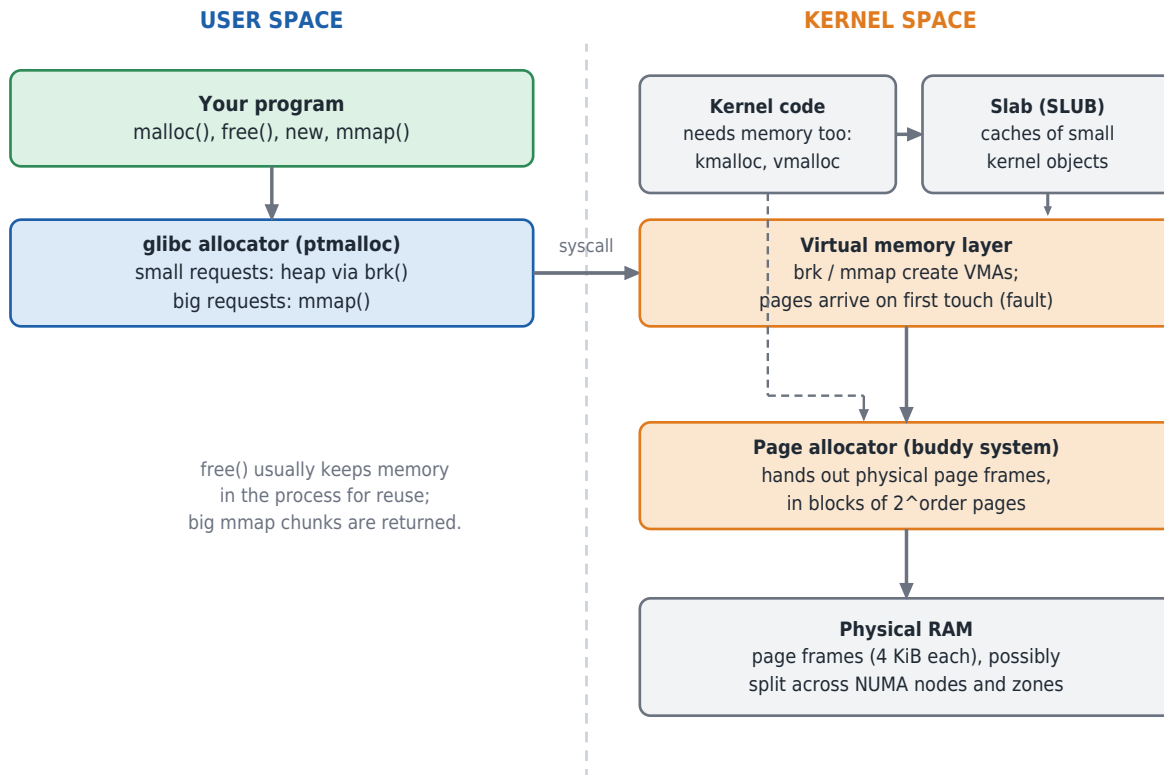


Figure 6.1: Layers of memory allocation, from a user's `malloc` to physical frames.

6.1 The slab allocator (SLUB)

A **slab cache** manages objects of one fixed size (or one kind of object). It obtains pages from the buddy allocator, cuts them into equal slots, and keeps free slots on lists. Allocation and free are then fast and waste almost nothing. Caching also lets the kernel keep frequently used objects warm, per CPU. The implementation used today is **SLUB**; it is the only slab allocator left in the kernel since Linux 6.8.

Kernel code uses `kmem_cache_alloc()` for a dedicated cache (one per common structure type), or `kmalloc()`, which picks from general-purpose caches named `kmalloc-64`, `kmalloc-128`, ..., `kmalloc-8k`. The cache list is in `/proc/slabinfo` (root only) and summarized by the `slabtop` command. Real output:

```
# name          <active_objs> <num_objs> <objsize> <objperslab> <pagesperslab> ...
inode_cache     195         195      616      13         2
dentry          1743        1743      192       21         1
vm_area_struct  273         273      192       21         1
task_struct     90          90     5888        5         8
mm_struct       10          10     1600       10         4
kmalloc-1k      344         344     1024        8         2
kmalloc-64     1088        1088        64       64         1
```

Read one row: `task_struct` objects are **5888 bytes** each; 5 fit in a slab made of 8 pages (32 KiB), and 90 exist because 90 processes and threads exist on that machine. `vm_area_struct` (a VMA) is only 192 bytes, 21 per page. Objects like `dentry` and `inode` (file system caches) are **reclaimable**: the kernel can free them when memory is short.

6.2 kmalloc vs vmalloc

	kmalloc(size, flags)	vmalloc(size)
Virtual memory	Contiguous	Contiguous
Physical memory	Contiguous	Scattered pages stitched together with page tables
Speed	Fast; uses the kernel's direct map (no extra page tables)	Slower; builds page table entries; more TLB pressure
Size	Small to moderate (a few MB at most)	Large blocks
Typical use	Most kernel objects; buffers for DMA	Big buffers, loaded kernel modules, large tables

6.3 The direct map

The kernel maps **all** of physical RAM into its own address space at a fixed offset (the "direct map"). Converting between a physical address and its kernel virtual address is then simple arithmetic, which makes `kmalloc` fast. It is also why pages given to `kmalloc` are physically contiguous.

6.4 Kernel memory is not free either

- **Kernel stacks**: each thread has a 16 KiB kernel stack (allocated with guard pages so overflow is detected).
- **Page tables** themselves consume memory: about 2 MiB of PTEs for every fully-mapped GiB of 4 KiB pages. They show as `PageTables` in `/proc/meminfo`.
- **Slab memory** appears as `Slab` (split into `SReclaimable` and `SUnreclaim`). **Shrinkers** let the kernel ask caches to give memory back when it is short.

CHAPTER 7

malloc() in User Space

When your program calls `malloc`, it is not calling the kernel. It is calling the C library's **allocator** (glibc's `ptmalloc`), a piece of user-space code that manages a pool of memory and only asks the kernel for more when the pool runs out.

7.1 Two ways to get memory from the kernel

- **The heap via `brk()`:** for small requests. glibc extends the heap VMA in chunks and carves out pieces.
- **`mmap()`:** for large requests. By default anything of about **128 KiB or more** gets its own anonymous mapping (the threshold adjusts dynamically up to 32 MiB). When you `free` it, the mapping is unmapped and the memory returns to the OS at once.

Lab 4 shows this with `strace`. A program calls `malloc(1000)`, `malloc(200 KiB)`, then `malloc(64 MiB)`:

```
brk(NULL)                = 0x565351626000          <- where does the heap end?
brk(0x565351647000)      = 0x565351647000          <- malloc(1000): heap grows 132 KiB
mmap(NULL, 208896, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f2a29bcd000
mmap(NULL, 67112960, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f2a25bcc000
munmap(0x7f2a25bcc000, 67112960)          = 0
munmap(0x7f2a29bcd000, 208896)            = 0
```

In order: `brk` finds and then grows the heap for `malloc(1000)`; the two `mmap` calls are the 200 KiB and 64 MiB blocks; the two `munmap` calls are their `free`s, which return the memory at once.

Notice that the first `malloc(1000)` grew the heap by 132 KiB, far more than requested, so later small allocations need no system call. Notice also that `free(a)` for the small block produced **no** system call at all.

7.2 Inside the allocator

- **Chunks and bins.** Freed chunks are kept on lists ("bins") sorted by size so the next `malloc` of that size can reuse them instantly. Recent small frees go to a per-thread **tcache**.
- **Arenas.** To reduce lock contention, multithreaded programs get several arenas (independent heaps).
- **`free()` rarely returns small memory to the OS.** Memory in the middle of the heap cannot be given back because the heap is one contiguous VMA. Only free space at the very top can be trimmed. So a process's RSS often does not shrink after it frees lots of small blocks.
- **`calloc` returns zeroed memory.** For fresh `mmap`'ed memory it skips the explicit zeroing, because the kernel already supplies zeros.

KEY IDEA

`malloc` returning a valid pointer does not mean the memory exists. On Linux, the address range is usually only a promise (Chapters 4 and 10). The real cost is paid when you first touch each page, and the program may be killed then, not at `malloc`.

7.3 Typical memory bugs

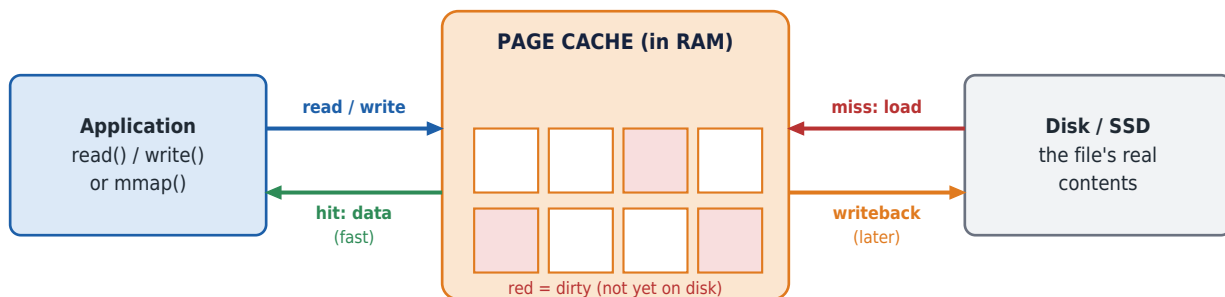
Bug	What happens	How to catch it
Memory leak	Allocated blocks never freed; RSS grows forever	<code>valgrind --leak-check=full</code> , AddressSanitizer
Use after free	Reading or writing a block after <code>free</code> ; corrupts the allocator or other data	AddressSanitizer, valgrind
Double free	Freeing the same pointer twice; allocator corruption or a crash	glibc detects some cases ("free(): double free detected"); ASan
Buffer overflow	Writing past the end of a block; overwrites neighbor chunk or metadata	AddressSanitizer, valgrind
Uninitialized read	<code>malloc</code> memory holds garbage (it is not zeroed)	valgrind, compiler warnings, <code>-fsanitize=memory</code> (clang)
Stack overflow	Deep recursion or huge local arrays hit the stack limit; SIGSEGV	Smaller frames, <code>ulimit -s</code> , heap allocation

CHAPTER 8

The Page Cache and File I/O

8.1 Using free RAM as a disk cache

Reading from disk is slow, so Linux keeps the contents of files it has read or written in RAM, in the **page cache**. When a program reads a file, the kernel first checks the cache. A hit is served from memory; a miss reads the pages from disk and leaves them in the cache for next time. The cache grows to fill otherwise unused RAM and shrinks instantly when programs need memory.



Reads are served from RAM when possible; writes return quickly and reach the disk later. `fsync()` forces a file's dirty pages out. The kernel also reads ahead when it sees sequential access.

Figure 8.1: Reads and writes go through the page cache.

8.2 Reads, writes, and writeback

- **Read:** `read()` copies data from cached pages to your buffer. On a miss, the kernel starts disk I/O; if the access pattern looks sequential it also **reads ahead** to fetch the following pages before you ask.
- **Write:** `write()` copies your data into cache pages and marks them **dirty**, then returns. It does not wait for the disk. This makes writes fast but means data can be lost in a power failure until it is flushed.
- **Writeback:** background kernel threads write dirty pages to disk when they are old enough or when too much memory is dirty. Real settings: `vm.dirty_background_ratio = 10` (start background writeback at 10% of memory dirty) and `vm.dirty_ratio = 20` (at 20%, writers themselves are forced to wait). Dirty data is normally flushed within about 30 seconds.
- `fsync(fd)` forces one file's dirty pages to disk and waits. Databases and editors use it to make changes durable. `O_DIRECT` bypasses the page cache entirely.

8.3 Memory-mapped files

`mmap` of a file makes file pages appear directly in your address space, backed by the same page cache. The first access to each page causes a fault: a **minor** fault if the page is already cached, a **major** fault if it must be read. There is no copy between kernel and user buffers. This is also how the kernel loads **programs and shared libraries**: their code pages are file-backed, so every process running `/bin/bash` or using `libc.so` shares one copy of those pages in the page cache.

8.4 Reading free

Because the cache uses free memory, **free** can look alarming to newcomers:

```
$ free -m
Mem:          total        used        free      shared  buff/cache   available
             3999         202       3889          13         52        3797
```

- **used:** memory in use by programs and the kernel (not counting reclaimable cache).
- **free:** completely unused RAM. It is *normal* for this to be small on a busy system.
- **buff/cache:** page cache plus buffers. Most of it can be dropped quickly if needed.
- **available:** the kernel's estimate of memory available to new programs *without* swapping. **This is the number to look at**, not **free**.

TRY IT

Lab 6 shows the cache filling: reading a 64 MB file raised **buff/cache** from 43 MB to 107 MB, and a second read of the file was about three times faster. On a real system with a slow disk the difference is far larger. (Root can force the cache empty for experiments with `echo 3 > /proc/sys/vm/drop_caches`. Never do this on a production system; it only slows things down.)

CHAPTER 9

Reclaim, Swap, and Page Replacement

9.1 When memory runs low

Because of the page cache, a healthy Linux system usually has very little truly free memory. The kernel therefore continually **reclaims** pages: it takes back pages that are least likely to be needed soon. A kernel thread called **kswapd** does this in the background when free memory falls below the **low** watermark (Figure 5.3). If a process needs memory faster than kswapd provides it, that process performs **direct reclaim** itself and stalls.

9.2 What can be reclaimed?

Kind of page	How to reclaim it	Cost
Clean file-backed page (in the page cache, matches disk)	Simply drop it. Read again from the file if needed.	Free
Dirty file-backed page	Write to its file first, then drop.	One disk write
Anonymous page (heap, stack)	Write to swap first, then drop. Impossible without swap space.	One disk write, and a read later if used again
Slab caches (dentries, inodes)	Shrinkers free unused objects.	Cheap
Pinned or locked pages (<code>mlock</code> , kernel pages)	Cannot be reclaimed.	n/a

9.3 Choosing victims: the LRU lists

Which pages should go? The ideal is the one that will be used furthest in the future, but the kernel cannot see the future. It approximates **LRU (least recently used)**: pages not used for a long time probably will not be used soon. Linux keeps **active** and **inactive** lists (separately for file and anonymous pages). New pages start on the inactive list; pages referenced again are promoted to active; reclaim takes from the tail of the inactive list.

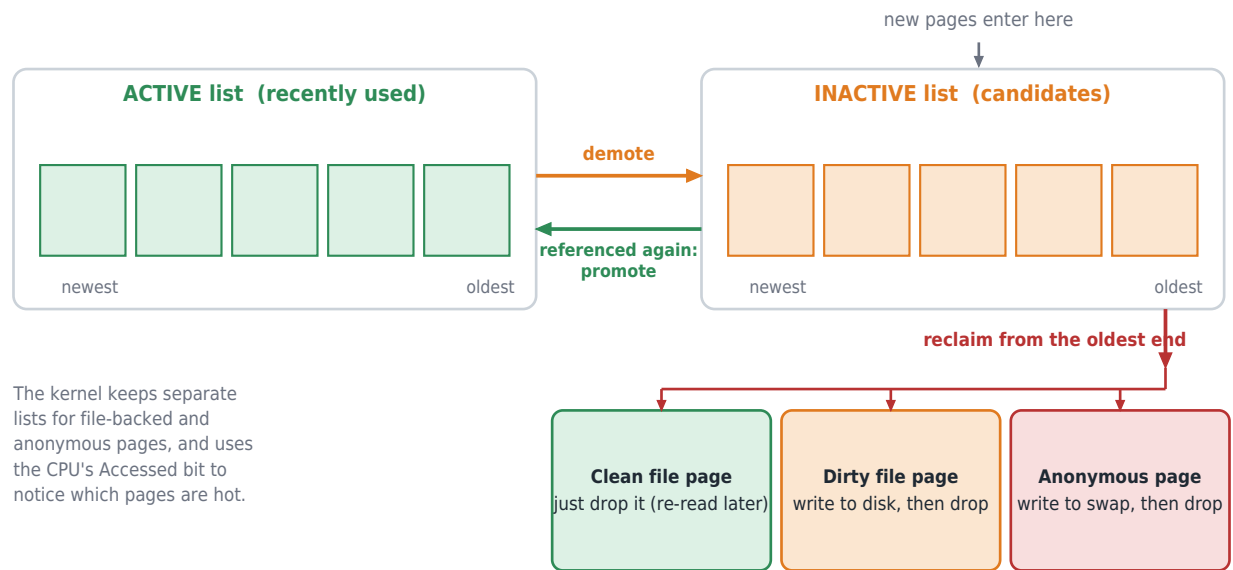


Figure 9.1: The active and inactive lists, and what reclaim does with each kind of page.

- **How it knows a page was used:** the CPU sets the **Accessed** bit in the PTE on every access. The kernel scans and clears these bits, using a reverse mapping (rmap) to find every PTE that maps a given page. A page with the bit set since the last scan was recently used.
- This is a **Clock**-style approximation (second chance) rather than exact LRU, which would require updating a list on every memory access, impossible in software.
- Linux 6.1 added an alternative, the **Multi-Gen LRU (MGLRU)**, with several age generations instead of two lists. It is an option in the kernel and the default on some distributions.

9.4 Swap

Swap is disk space used to hold anonymous pages that do not fit in RAM. It can be a partition or a file (**swapon**, **mkswap**); compressed in-RAM variants (**zram**, **zswap**) are common on desktops and phones.

- 1 Reclaim picks a cold anonymous page and writes it to a free swap slot.
 - 2 The kernel changes the page's PTE: Present = 0, and the other bits record *which swap slot* holds it.
 - 3 The frame is freed for other use.
 - 4 If the process touches the page later, it faults (a **major** fault); **do_swap_page()** reads it back from swap, allocates a frame, and updates the PTE.
- **vm.swappiness** (60 by default, range 0 to 200) controls the balance between reclaiming file cache and swapping anonymous pages. Higher values swap more eagerly. Lower values prefer dropping cache.
 - **Thrashing.** If the total **working set** (pages processes actively use) exceeds RAM, pages are evicted and immediately needed again. The system spends its time waiting for disk instead of doing work.
 - **Pressure Stall Information (PSI)** shows how much time tasks stall waiting for memory:
`/proc/pressure/memory` printed `some avg10=0.00 ...` on the idle test machine. Values above a few percent suggest real memory pressure. (Note: the test machine had no swap at all, **SwapTotal: 0 kB**, so only file pages could be reclaimed.)

9.5 Textbook page-replacement algorithms

Operating-systems courses study several policies. Here is how each behaves on the classic reference string **7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1** with **3 frames**. (F marks a page fault.)

Policy	Idea	Implementable?
FIFO	Evict the page that has been in memory longest	Trivial, but ignores usage; can evict hot pages; suffers Belady's anomaly
OPT (optimal)	Evict the page whose next use is furthest in the future	Impossible (needs the future); used as a benchmark
LRU	Evict the page unused for the longest time	Good, but exact tracking is costly in hardware or software
Clock (second chance)	Sweep a hand over frames; give a page with the reference bit set another chance, clear the bit; evict the first page with it clear	Cheap approximation of LRU. This is the family Linux belongs to

FIFO with 3 frames: 15 page faults

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
Frame 2		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
Frame 3			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
Fault?	F	F	F	F		F	F	F	F	F	F			F	F			F	F	F

LRU with 3 frames: 12 page faults

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
Frame 2		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
Frame 3			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
Fault?	F	F	F	F		F		F	F	F	F			F		F		F		

OPT gets **9** faults on the same string. The order is OPT (9) < LRU (12) < FIFO (15); no practical policy can beat OPT, and LRU-like policies usually beat FIFO because programs show **locality of reference**: recently used pages are likely to be used again.

Belady's anomaly

More frames should mean fewer faults, but for FIFO that is not guaranteed. On the reference string **1 2 3 4 1 2 5 1 2 3 4 5**, FIFO has **9 faults with 3 frames** but **10 faults with 4 frames**. LRU and OPT never do this (they are "stack algorithms"): with LRU the same string gives 10 faults with 3 frames and 8 with 4.

CHAPTER 10

Overcommit and the OOM Killer

10.1 Allocating more than you have

Because allocation is lazy, the kernel can promise more memory than physically exists. This is called **overcommit**, and by default Linux allows a generous amount of it. Lab 2 demonstrates it on the 4 GB test machine:

```
start          VmSize=    2688 kB  VmRSS=    1372 kB
after malloc(1 GiB)  VmSize= 1051268 kB  VmRSS=    1696 kB    <- 1 GiB promised
after touching 100 MiB  VmSize= 1051268 kB  VmRSS= 104096 kB    <- ~100 MiB real
```

Why allow this? Because much allocated memory is never touched: huge sparse arrays, buffers sized for the worst case, and above all `fork()`, which momentarily *doubles* a large process's committed memory on paper even though copy-on-write means almost nothing is really copied. Strict accounting would reject many legitimate programs.

10.2 The three overcommit modes

vm.overcommit_memory	Behavior
0 (heuristic, the default)	Refuse only <i>obviously</i> excessive single requests (larger than total RAM plus swap). Otherwise allow.
1 (always)	Never refuse. Useful for some scientific programs with huge sparse allocations.
2 (strict)	Refuse allocations that would push total committed memory over <code>CommitLimit</code> = swap + RAM x <code>overcommit_ratio</code> / 100.

The test machine used mode 0, `overcommit_ratio` 50, and no swap: `CommitLimit` in `/proc/meminfo` read 2,047,656 kB, which is 50% of its 4,095,316 kB of RAM (the limit only applies in mode 2). `Committed_AS` (307,852 kB) is how much memory has been promised in total.

10.3 When the promise is called in: the OOM killer

If processes actually touch more memory than exists, and reclaim (dropping cache, swapping) cannot free enough, the kernel has a problem: it has already promised memory it cannot supply. As a last resort it invokes the **OOM (out-of-memory) killer**, which chooses a process and terminates it with `SIGKILL` to free its memory.

- **How it chooses:** each process gets a **badness score** in `/proc/<PID>/oom_score`, mainly proportional to the share of RAM (plus swap and page tables) it uses. The highest score is killed. Large, memory-hungry processes are the likeliest victims, not necessarily the one that triggered the problem.
- **Adjusting it:** write to `/proc/<PID>/oom_score_adj` (range -1000 to +1000). -1000 exempts a process entirely; +1000 makes it the preferred victim. Important daemons are often protected this way.
- **Evidence:** the kernel log records it. `dmesg | grep -i "out of memory"` shows lines such as "Out of memory: Killed process 1234 (name)". The victim's exit status shows as **137** (128 + 9, killed by SIGKILL) in a shell.

- **cgroups:** in a container or systemd service with `memory.max` set, hitting the limit triggers an OOM kill **inside that group** without touching the rest of the system.
- **User-space helpers** such as `systemd-oomd` and `earlyoom` act earlier, using pressure information, to kill something before the machine stalls.

WATCH OUT

A process can be killed at a moment unrelated to its `malloc` call, since memory is committed only when touched. Programs cannot rely on `malloc` returning NULL to detect low memory on a default Linux system. Code that must handle memory exhaustion needs other strategies (limits, `mlock`, preallocating and touching, or strict overcommit).

CHAPTER 11

Advanced Topics

11.1 Huge pages

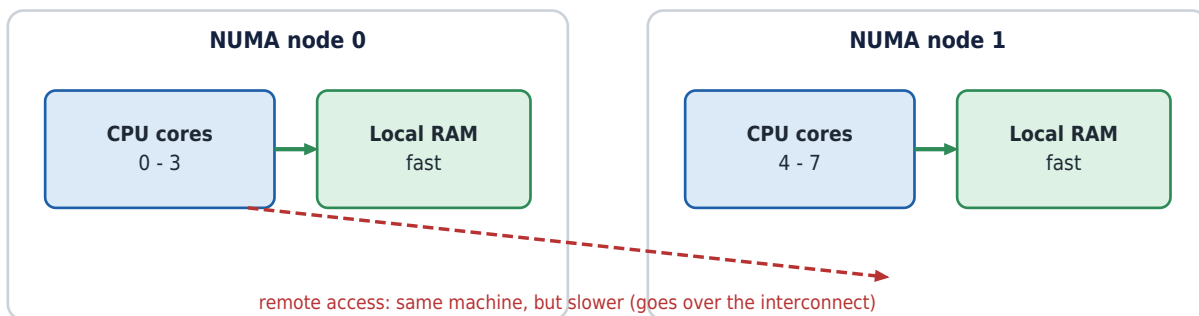
As Section 2.6 explained, 2 MiB pages reduce page-table size and TLB misses, which helps large-memory programs such as databases and JVMs. Linux offers two ways:

- **Explicit huge pages (`hugetlbfs`):** an administrator reserves a pool (`vm.nr_hugepages`); programs request them with `mmap(MAP_HUGETLB)`. Predictable but manual. (`HugePages_Total: 0` on the test machine.)
- **Transparent Huge Pages (THP):** the kernel automatically uses 2 MiB pages where it can, and a thread called `khugepaged` merges small pages into huge ones in the background. The setting lives in `/sys/kernel/mm/transparent_hugepage/enabled: always, madvise` (only for regions that ask with `madvise(MADV_HUGEPAGE)`), or `never`. The test machine showed `madvise`.

Trade-offs: huge pages need 2 MiB of contiguous memory (harder as memory fragments), can waste memory for sparse use, and can cause latency spikes from compaction. Some latency-sensitive systems disable THP.

11.2 NUMA

Large servers have several **NUMA nodes**: each CPU socket has its own local memory, and reaching another socket's memory is slower. The buddy allocator keeps free lists per node and zone.



Linux prefers to allocate memory on the node of the CPU that first touches it.
`numactl` and `mbind()` let you override that policy.

Figure 11.1: Local and remote memory in a NUMA machine.

- The default policy is to allocate on the node of the CPU that **first touches** a page.
- `numactl --cpunodebind=0 --membind=0 cmd` and `mbind()/set_mempolicy()` override the policy.
- `numastat` and `/sys/devices/system/node/` show per-node usage. The test machine had only `node0`, so it behaves like an ordinary uniform machine.

11.3 Pinning memory and shared memory

- `mlock()` / `mlockall()` pin pages in RAM so they are never swapped or reclaimed. Used for real-time programs and secrets like encryption keys. Limited by `ulimit -l` (8192 KB on the test machine).
- **Shared memory:** `shm_open` + `mmap`, System V `shmget`, and `tmpfs (/dev/shm)` let unrelated processes map the same pages. It appears as `Shmem` in `/proc/meminfo` (13,380 kB on the test machine). Shared memory needs its own synchronization.
- **KSM (Kernel Same-page Merging)** scans for identical pages and merges them (with COW). Useful for hosts running many similar virtual machines.

11.4 Memory cgroups

Control groups can limit and account memory for a set of processes. With cgroup v2, under `/sys/fs/cgroup`:

File	Meaning
<code>memory.max</code>	Hard limit. Beyond it the group reclaims, then the OOM killer runs inside the group.
<code>memory.high</code>	Soft limit: above it the group is throttled and forced to reclaim.
<code>memory.current</code>	Memory currently used by the group (including page cache).
<code>memory.swap.max</code>	Swap the group may use.

11.5 Memory protection features

Feature	What it does
Permissions + NX (W^X)	Pages are writable or executable, not both by default. Blocks classic code injection.
ASLR / KASLR	Random layout for user space and for the kernel itself.
Guard pages and gaps	Inaccessible pages beside stacks; overflow becomes a fault.
SMEP / SMAP	CPU features preventing the kernel from executing (SMEP) or accidentally reading/writing (SMAP) user memory.
KPTI	Separates user and kernel page tables to mitigate Meltdown, at some performance cost.
Stack protector, hardened allocators	Compiler and library defenses (canaries, safe-unlinking) against overflows.

CHAPTER 12

Observing Memory

12.1 The tools

Tool	What it shows
<code>free -h</code>	System-wide summary: used, free, cache, available, swap
<code>/proc/meminfo</code>	Detailed system-wide counters (below)
<code>vmstat 1</code>	Live memory, swap, I/O, and CPU per second (si/so = swap in/out)
<code>top, htop</code>	Per-process VIRT, RES, SHR, and %MEM
<code>ps -o pid,rss,vsz,min_flt,maj_flt,cmd</code>	Per-process sizes and fault counts
<code>/proc/PID/status</code>	VmSize, VmRSS, VmData, VmStk, VmSwap, ...
<code>/proc/PID/smmaps, smmaps_rollup, pmap -x PID</code>	Per-VMA and total RSS, PSS, shared and private, dirty
<code>/proc/PID/maps</code>	Layout of the address space (Chapter 3)
<code>/proc/vmstat</code>	Kernel-wide counters: pgfault, pgmajfault, pswpin, pswpout, oom_kill, ...
<code>slabtop, /proc/slabinfo</code>	Kernel object caches
<code>/proc/buddyinfo, /proc/zoneinfo</code>	Free lists and watermarks (Chapter 5)
<code>/proc/pressure/memory</code>	Memory pressure (PSI)
<code>dmesg</code>	OOM-killer reports and memory errors
<code>valgrind, AddressSanitizer</code>	Find leaks and bugs in your program's own use of memory

12.2 Key fields of /proc/meminfo

Real values from the test machine (4 GB RAM):

```

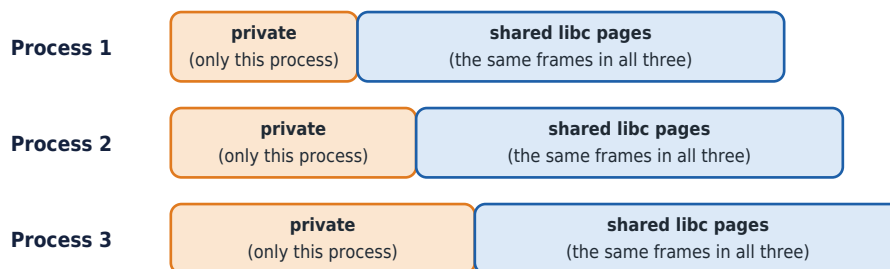
MemTotal:      4095316 kB    total usable RAM
MemFree:       3982368 kB    completely unused
MemAvailable:  3887944 kB    estimate of what programs can get without swapping
Buffers:       3168 kB      block-device metadata cache
Cached:        47824 kB      page cache (file data)
AnonPages:     11176 kB      anonymous memory mapped by processes (heap, stack)
Mapped:        31788 kB      file pages that processes have mapped
Shmem:         13380 kB      shared memory and tmpfs
Slab:          14116 kB      kernel object caches (SReclaimable 2512, SUnreclaim 11604)
KernelStack:   1248 kB      kernel stacks
PageTables:    1280 kB      memory used by page tables
Dirty:         108 kB       file data waiting to be written to disk
SwapTotal:     0 kB         no swap on this machine
CommitLimit:   2047656 kB   only enforced in overcommit mode 2
Committed_AS:  307852 kB    memory promised so far

```

12.3 Per-process sizes: VSZ, RSS, PSS, USS

Measure	Meaning	Pitfall
VSZ / VmSize	Total virtual address space mapped	Includes memory never touched, so usually huge and misleading
RSS / VmRSS	Pages currently resident in RAM (anonymous + file-backed + shared)	Counts shared pages fully in every process, so summing RSS over-counts
PSS	Proportional set size: private pages + shared pages divided by the number of sharers	None; sum of PSS is the true total
USS	Unique set size: only pages private to the process	What would be freed by killing it
VmSwap	Anonymous pages of the process currently in swap	

Three processes share one copy of libc in RAM



RSS counts every resident page a process maps, including shared ones. Summing RSS over-counts.
PSS (proportional) = private + shared / number of sharers. Summing PSS gives the true total.
USS (unique) = private pages only: what you would free by killing the process.

Figure 12.1: Why summing RSS across processes over-counts shared memory.

The kernel exposes these in `/proc/<PID>/smaps_rollup`. Real output for a plain `sleep` process:

```
Rss:          1692 kB      resident pages, shared or not
Pss:           560 kB      proportional: only a third of the libc pages are "its"
Shared_Clean: 1564 kB      mostly libc code, shared with other processes
Private_Dirty: 100 kB      its own heap and stack (this is essentially its USS)
Anonymous:    100 kB
```

A tiny `sleep` process has an RSS of 1.7 MB, but its PSS is only 0.56 MB, because most of what it maps is shared library code that the page cache holds once for every process using it.

CHAPTER 13

Hands-On Labs

Try these on any Linux machine (a VM or WSL2 works). Compile with `gcc file.c -o name`. Outputs shown were captured on a real system.

Lab 1: Demand paging, one fault per page

Goal: watch memory get allocated lazily. The program maps 10 MiB, then touches it in stages while printing the resident set size (RSS), the number of minor faults, and whether a page is present according to

`/proc/self/pagemap`.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <fcntl.h>
#include <stdint.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/resource.h>

#define NPAGES 2560 /* 2560 x 4 KiB = 10 MiB */

static long rss_kb(void) {
    FILE *f = fopen("/proc/self/status", "r");
    char line[256]; long kb = -1;
    while (fgets(line, sizeof line, f))
        if (sscanf(line, "VmRSS: %ld kB", &kb) == 1) break;
    fclose(f);
    return kb;
}

static long minor_faults(void) {
    struct rusage ru;
    getrusage(RUSAGE_SELF, &ru);
    return ru.ru_minflt;
}

static int page_present(int pm_fd, void *addr) { /* bit 63 of the pagemap entry = present */
    uint64_t e;
    pread(pm_fd, &e, 8, ((uintptr_t)addr / 4096) * 8);
    return (int)(e >> 63) & 1;
}

int main(void) {
    long pg = sysconf(_SC_PAGESIZE);
    int pm = open("/proc/self/pagemap", O_RDONLY);
    char *buf = mmap(NULL, NPAGES * pg, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    printf("after mmap : RSS=%6ld kB minor faults=%ld page 0 present=%d\n",
           rss_kb(), minor_faults(), page_present(pm, buf));

    long f0 = minor_faults();
    buf[0] = 1; /* touch ONE page */
    printf("touched 1 : RSS=%6ld kB new faults=%ld page 0 present=%d page 1 present=%d\n",
           rss_kb(), minor_faults() - f0, page_present(pm, buf), page_present(pm, buf + pg));

    f0 = minor_faults();
    for (long i = 0; i < NPAGES / 2; i++) buf[i * pg] = 1;
    printf("touched half: RSS=%6ld kB new faults=%ld\n", rss_kb(), minor_faults() - f0);

    f0 = minor_faults();
    for (long i = 0; i < NPAGES; i++) buf[i * pg] = 1;
    printf("touched all : RSS=%6ld kB new faults=%ld\n", rss_kb(), minor_faults() - f0);
    return 0;
}
```

```
}
```

```
after mmap : RSS= 1632 kB  minor faults=60  page 0 present=0
touched 1  : RSS= 1760 kB  new faults=1  page 0 present=1  page 1 present=0
touched half: RSS= 6876 kB  new faults=1279
touched all : RSS= 11996 kB  new faults=1280
```

Observe: mapping 10 MiB added **no** RSS and page 0 was not present. Touching one page caused exactly **one** fault and made only that page present. Touching the first half cost 1279 more faults (one page was already there), and the second half another 1280: **one fault per 4 KiB page**, and RSS grew by about 10 MiB in total.

CHECK YOURSELF

Change the program to only *read* `buf[i*pg]` instead of writing. Does RSS grow? (Hint: Section 4.4, the shared zero page.) Then try `MAP_POPULATE` in the flags. What changes?

Lab 2: Overcommit, the promise vs the reality

Goal: see that `malloc` of a huge block costs almost nothing until the memory is touched.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void show(const char *label) {
    FILE *f = fopen("/proc/self/status", "r");
    char line[256]; long vsz = 0, rss = 0;
    while (fgets(line, sizeof line, f)) {
        sscanf(line, "VmSize: %ld kB", &vsz);
        sscanf(line, "VmRSS: %ld kB", &rss);
    }
    fclose(f);
    printf("%-26s VmSize=%9ld kB  VmRSS=%7ld kB\n", label, vsz, rss);
}

int main(void) {
    size_t size = 1UL << 30; /* 1 GiB */
    show("start");
    char *p = malloc(size);
    if (!p) { puts("malloc failed"); return 1; }
    show("after malloc(1 GiB)");
    memset(p, 1, 100UL << 20); /* touch only 100 MiB */
    show("after touching 100 MiB");
    return 0;
}
```

```
start          VmSize=    2688 kB  VmRSS=    1372 kB
after malloc(1 GiB) VmSize=  1051268 kB  VmRSS=    1696 kB
after touching 100 MiB VmSize=  1051268 kB  VmRSS=  104096 kB
```

Observe: `VmSize` jumped by 1 GiB at `malloc`, but `VmRSS` barely moved; after touching 100 MiB, RSS grew by about 100 MiB. **Experiment:** compare with `cat /proc/sys/vm/overcommit_memory`. Do **not** raise the size beyond your machine's RAM and touch it all, or you may trigger the OOM killer.

Lab 3: Copy-on-write after fork, measured with PSS

Goal: watch pages become shared at **fork** and get copied when written, using `/proc/self/smaps_rollup`.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (64UL << 20)                                /* 64 MiB */

static void show(const char *who, const char *when) {
    FILE *f = fopen("/proc/self/smaps_rollup", "r");
    char line[256]; long rss = 0, pss = 0, shd = 0, prd = 0;
    while (fgets(line, sizeof line, f)) {
        sscanf(line, "Rss: %ld kB", &rss);
        sscanf(line, "Pss: %ld kB", &pss);
        sscanf(line, "Shared_Dirty: %ld kB", &shd);
        sscanf(line, "Private_Dirty: %ld kB", &prd);
    }
    fclose(f);
    printf("%-6s %-19s Rss=%6ld Pss=%6ld Shared_Dirty=%6ld Private_Dirty=%6ld\n",
        who, when, rss, pss, shd, prd);
    fflush(stdout);
}

int main(void) {
    char *p = malloc(SIZE);
    memset(p, 1, SIZE);                                /* parent touches all 64 MiB */
    show("parent", "before fork");
    pid_t pid = fork();
    if (pid == 0) {
        show("child", "just after fork");
        memset(p, 2, SIZE / 2);                        /* child rewrites half of it */
        show("child", "after writing half");
        _exit(0);
    }
    waitpid(pid, NULL, 0);
    show("parent", "after child exited");
    return 0;
}
```

```
parent before fork      Rss= 66984 Pss= 66122 Shared_Dirty=    0 Private_Dirty= 65652
child just after fork   Rss= 66324 Pss= 33031 Shared_Dirty= 65616 Private_Dirty=   36
child after writing half Rss= 66640 Pss= 49517 Shared_Dirty= 32840 Private_Dirty= 32812
parent after child exited Rss= 67416 Pss= 66340 Shared_Dirty=    0 Private_Dirty= 65656
(all values in kB)
```

Observe: just after **fork**, the child's RSS is 64 MiB, but nearly all of it is **Shared_Dirty** and its PSS is **half** (33,031 kB): each of the two processes is charged for half of every shared page. After the child rewrites half of the data, roughly half of its pages become **Private_Dirty**: the kernel copied them (32,812 kB) on the write faults. PSS rose to 49,517 kB. Once the child exits, the parent again owns everything privately. Note how **RSS hid all of this**; PSS and the shared/private split tell the real story.

Lab 4: malloc, brk, and mmap

```
#include <stdlib.h>
int main(void) {
    void *a = malloc(1000);                            /* small */
    void *b = malloc(200 * 1024);                       /* 200 KiB: above the 128 KiB mmap threshold */
    void *c = malloc(64 * 1024 * 1024);                 /* 64 MiB */
    free(c); free(b); free(a);
    return 0;
}
```

```
$ strace -e trace=brk,mmap,munmap ./lab4
brk(NULL)           = 0x565351626000
brk(0x565351647000) = 0x565351647000
mmap(NULL, 208896, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f2a29bcd000
mmap(NULL, 67112960, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f2a25bcc000
munmap(0x7f2a25bcc000, 67112960)      = 0
munmap(0x7f2a29bcd000, 208896)        = 0
```

Observe: the small allocation used `brk`; the two large ones each got their own `mmap` and were returned with `munmap` at `free`. **Experiment:** allocate many 100-byte blocks in a loop and count `brk` calls; then free every other block and check RSS in `/proc/self/status`. Does it shrink?

Lab 5: The two kinds of segmentation fault

Goal: distinguish an unmapped address from a permission violation, using `SA_SIGINFO`.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>
#include <sys/mman.h>

static void handler(int sig, siginfo_t *si, void *ctx) {
    const char *why = "other\n";
    if (si->si_code == SEGV_MAPERR) why = "SEGV_MAPERR (address not mapped)\n";
    if (si->si_code == SEGV_ACCERR) why = "SEGV_ACCERR (mapped, but permission denied)\n";
    write(1, "  -> SIGSEGV: ", 14);
    write(1, why, strlen(why));
    _exit(1);
}

int main(int argc, char **argv) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_sigaction = handler;
    sa.sa_flags = SA_SIGINFO;
    sigaction(SIGSEGV, &sa, NULL);

    int which = argc > 1 ? atoi(argv[1]) : 0;
    long pg = sysconf(_SC_PAGESIZE);
    if (which == 0) {
        printf("case 0: write to address 0x10 (nothing mapped there)\n"); fflush(stdout);
        *(volatile char *)0x10 = 1;
    } else if (which == 1) {
        char *p = mmap(NULL, pg, PROT_NONE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
        printf("case 1: read a PROT_NONE page at %p\n", (void *)p); fflush(stdout);
        volatile char c = *p; (void)c;
    } else {
        char *p = mmap(NULL, pg, PROT_READ, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
        printf("case 2: write to a read-only page at %p\n", (void *)p); fflush(stdout);
        *(volatile char *)p = 1;
    }
    return 0;
}
```

```
$ ./lab5 0 ; ./lab5 1 ; ./lab5 2
case 0: write to address 0x10 (nothing mapped there)
-> SIGSEGV: SEGV_MAPERR (address not mapped)
case 1: read a PROT_NONE page at 0x7f0e13799000
-> SIGSEGV: SEGV_ACCERR (mapped, but permission denied)
case 2: write to a read-only page at 0x7f76fa8c4000
-> SIGSEGV: SEGV_ACCERR (mapped, but permission denied)
```

Observe: these are the two "invalid" branches of Figure 4.1: no VMA covers the address (`SEGV_MAPERR`), or a VMA exists but forbids the access (`SEGV_ACCERR`). The handler uses only `write` and `_exit`, both async-signal-safe. **Challenge:** extend the handler to print `si->si_addr` (the faulting address) and use `mprotect` to make a page writable from inside the handler so the program continues.

Lab 6: Watching the page cache

Goal: see file data enter the page cache, and see re-reads get faster. Needs root for `drop_caches`; use a scratch machine or VM.

```
$ dd if=/dev/urandom of=big.bin bs=1M count=64 status=none ; sync
$ echo 3 > /proc/sys/vm/drop_caches      # empty the page cache (root; experiments only)
$ free -m
Mem:           total        used        free        shared  buff/cache   available
              3999          208        3888           13           43        3790
$ cat big.bin > /dev/null
$ free -m
Mem:           total        used        free        shared  buff/cache   available
              3999          240        3824           13          107        3759

$ echo 3 > /proc/sys/vm/drop_caches
cold read: 27 ms      (read from disk)
warm read:  9 ms      (served from the page cache)
warm read:  9 ms
```

Observe: `buff/cache` grew by 64 MB (the file's size), while `used` barely changed: the file's data lives in the cache, not in any process. The warm reads were about three times faster. (The virtual disk here is fast; on slower disks the gap can be far larger.) **Experiment:** `dd if=/dev/zero of=out.bin bs=1M count=200` and watch `Dirty` in `/proc/meminfo` climb and then drop as writeback runs; then run `sync`.

CHAPTER 14

Review, Misconceptions, and Reference

14.1 Common misconceptions

Misconception	Reality
" <code>malloc</code> gives me RAM."	It gives you address space. Physical pages arrive on first touch, one 4 KiB page at a time.
"Free memory should be large; a small <code>free</code> number means trouble."	Linux uses spare RAM as page cache. Look at <code>available</code> , not <code>free</code> .
"A page fault is an error."	Most are normal: demand paging, copy-on-write, and page cache mapping. Only invalid faults become <code>SIGSEGV</code> .
" <code>fork</code> copies all of the parent's memory."	It copies page tables and marks pages copy-on-write. Data is copied later, per page, on write.
"RSS is how much memory a process really uses."	RSS double-counts shared pages. Use PSS or USS to compare.
"Virtual size (VSZ) is memory used."	It is address space mapped. Much of it may never be touched.
"Swap means the system is out of memory."	Swap may hold long-idle pages to leave RAM for cache. Heavy, continuous swap-in and swap-out (thrashing) is the problem.
" <code>free()</code> returns memory to the OS."	Only large (mmap'ed) blocks and the very top of the heap are returned. Small frees usually stay in the process.
" <code>malloc</code> returns NULL when memory runs out."	With default overcommit it almost never does; the OOM killer acts later, at touch time.
"The OOM killer kills the process that asked for too much."	It kills the process with the highest badness score, usually the largest, which may be innocent.
"Kernel memory is unlimited."	Page tables, slab caches, and stacks all use RAM; large machines pay real overhead.

14.2 Where to find it in the source

Paths may shift between kernel versions; use them as starting points.

Topic	File / function
Core structures	<code>include/linux/mm_types.h</code> : <code>struct page</code> , <code>struct folio</code> , <code>struct mm_struct</code> , <code>struct vm_area_struct</code>
Page fault entry (x86)	<code>arch/x86/mm/fault.c</code> : <code>do_user_addr_fault()</code>
Generic fault handling	<code>mm/memory.c</code> : <code>handle_mm_fault()</code> , <code>do_anonymous_page()</code> , <code>do_wp_page()</code> , <code>do_swap_page()</code>
File-backed faults, page cache	<code>mm/filemap.c</code> : <code>filemap_fault()</code> , <code>filemap_read()</code>
<code>mmap</code> , <code>brk</code> , VMAs	<code>mm/mmap.c</code> : <code>do_mmap()</code> , <code>do_brk_flags()</code> , <code>mm/mprotect.c</code>
Buddy / page allocator	<code>mm/page_alloc.c</code> : <code>__alloc_pages()</code> , <code>free_one_page()</code> ; <code>mm/compaction.c</code>

Topic	File / function
Slab (SLUB)	mm/slub.c: kmem_cache_alloc(), __kmalloc()
vmalloc	mm/vmalloc.c
Reclaim and LRU	mm/vmscan.c: kswapd(), shrink_node(); mm/swap.c; mm/workingset.c
Swap	mm/swapfile.c, mm/page_io.c, mm/swap_state.c
Writeback	mm/page-writeback.c, fs/fs-writeback.c
OOM killer	mm/oom_kill.c: out_of_memory(), oom_badness()
Huge pages	mm/huge_memory.c, mm/khugepaged.c, fs/hugetlbfs/
NUMA policy	mm/mempolicy.c
Memory cgroups	mm/memcontrol.c
/proc memory files	fs/proc/task_mmu.c (maps, smaps), fs/proc/meminfo.c

14.3 Review questions

- 1 Explain the difference between a virtual address and a physical address. Why do processes use virtual addresses?
- 2 Split the virtual address `0x00007f57d7a01234` into its page number and offset for 4 KiB pages. Which part is translated?
- 3 Why do page tables use several levels instead of one flat table? Calculate the size of a flat table for a 48-bit address space with 8-byte entries.
- 4 What is the TLB, and what happens on a TLB miss? Why is changing a mapping sometimes expensive on a multicore machine?
- 5 What is the difference between a VMA and a page table entry? Can a valid address have no PTE?
- 6 Describe the four kinds of `mmap` mapping (private/shared, file/anonymous) and give one use of each.
- 7 Explain demand paging. Why does `malloc(1 GiB)` succeed instantly and barely change RSS?
- 8 Distinguish minor, major, and invalid page faults with an example of each. Which are normal?
- 9 Describe how copy-on-write works after `fork`. What is copied at `fork` time and what is copied later?
- 10 How does the buddy allocator satisfy an order-2 request when only an order-4 block is free? What happens when it is freed?
- 11 What are the min, low, and high watermarks? Who reclaims memory in each region, and how does that affect the application?
- 12 Why does the kernel need a slab allocator on top of the page allocator? Contrast `kmalloc` and `vmalloc`.
- 13 In glibc, when does `malloc` use `brk` and when `mmap`? Why does freeing small blocks often not shrink the process?
- 14 What is the page cache? Explain read hits, read misses, dirty pages, writeback, and `fsync`.
- 15 Why is `MemAvailable` a better measure than `MemFree`?

- 16 What kinds of pages can be reclaimed, and what does each cost? Why can anonymous pages not be reclaimed without swap?
- 17 For the reference string 1 2 3 2 4 1 5 2 1 2 3 4 5 with 3 frames, count the page faults for FIFO, LRU, and OPT.
- 18 What is Belady's anomaly? Give the reference string and fault counts for FIFO with 3 and 4 frames.
- 19 What is overcommit? Describe the three modes and explain why the default allows it.
- 20 How does the OOM killer choose a victim? How can you protect a process from it? What exit status does the victim show?
- 21 Why does summing RSS over all processes overstate memory use? What should you use instead?
- 22 What are huge pages and what do they save? Give one advantage and one disadvantage of THP.

14.4 Answer key

- 1 A virtual address is what a program uses; a physical address is a real location in RAM. Virtual addresses give each process a private, contiguous space, isolation, sharing, and the ability to exceed RAM, because the kernel can map pages anywhere or nowhere.
- 2 Offset = low 12 bits = 0x234; virtual page number = 0x7f57d7a01. Only the page number is translated to a frame number; the offset is kept.
- 3 A flat table needs an entry for every page: $2^{48} / 2^{12} = 2^{36}$ entries $\times$ 8 bytes = 2^{39} bytes = 512 GiB per process. A multi-level tree allocates lower-level tables only for regions in use, so sparse address spaces stay small.
- 4 The TLB caches recent translations. On a miss, hardware walks the page tables (up to four memory reads) and refills it. Changing a mapping requires invalidating stale TLB entries on every CPU that might have them, using inter-processor interrupts (TLB shutdown).
- 5 A VMA is the kernel's record that a range is valid and what it means; a PTE is the current hardware mapping of one page. Yes: with demand paging, a page inside a VMA has no PTE until first touched.
- 6 Private file: program code and libraries (writes are copy-on-write). Shared file: memory-mapped file I/O. Private anonymous: heap, stack, big **malloc**. Shared anonymous: shared memory between related processes (or tmpfs).
- 7 Memory is allocated when a page is first touched, not when it is mapped. **malloc(1 GiB)** only creates a VMA (and possibly grows VmSize); no frames are allocated, so RSS barely moves until pages are used.
- 8 Minor: first touch of anonymous memory, file page already cached, COW copy (no disk). Major: needs disk I/O, e.g. swapped-out or uncached file page. Invalid: address in no VMA or permission violation, giving SIGSEGV. Minor and major faults are normal.
- 9 At **fork**, page tables are copied and writable pages are marked read-only and shared (refcount 2). A write faults; the kernel then copies just that page for the writer. Untouched pages are never copied.
- 10 Take the order-4 block, split into two order-3 blocks, keep one, split it into two order-2 blocks, return one, and put the leftover order-3 and order-2 halves on their free lists. On free, if its buddy is free the blocks merge to order 3, and so on upward.
- 11 min, low, high are per-zone free-page thresholds. Above high: nothing. Below low: kswapd reclaims in the background (application unaffected). Below min: allocating processes perform direct reclaim and stall, so the application slows down.
- 12 The page allocator hands out whole pages, wasteful for small objects. Slab caches carve pages into fixed-size objects with little waste and fast reuse. **kmalloc** returns physically contiguous memory (fast, direct map); **vmalloc** returns virtually contiguous but physically scattered memory (for large buffers).
- 13 Small requests (below about 128 KiB) come from the **brk** heap; large ones use **mmap**. Small frees go back to the allocator's bins, not the OS, since the heap is one contiguous region and only the top can be trimmed.
- 14 The page cache holds file data in RAM. A read hit is served from cache; a miss reads from disk and caches it. Writes copy data into cache pages marked dirty; writeback threads flush them later; **fsync** forces one file's dirty pages to disk.
- 15 **MemFree** ignores reclaimable cache, so it is small on a healthy system. **MemAvailable** estimates memory usable by new programs without swapping, including reclaimable cache.

- 16 Clean file pages (drop, free), dirty file pages (write then drop), anonymous pages (write to swap then drop), slab objects (shrinkers). Anonymous pages have no backing file, so without swap there is nowhere to put their contents.
- 17 FIFO: 10 faults. LRU: 10 faults. OPT: 7 faults.
- 18 With FIFO, more frames can cause more faults. On 1 2 3 4 1 2 5 1 2 3 4 5, FIFO gives 9 faults with 3 frames and 10 with 4 frames.
- 19 Overcommit lets the kernel promise more memory than exists, because much of it is never touched. Mode 0: refuse only obviously excessive requests (default). Mode 1: always allow. Mode 2: enforce `CommitLimit`. Default allows it so fork, sparse arrays, and oversized buffers work.
- 20 It computes a badness score (mostly memory share) and kills the highest. Protect with `oom_score_adj = -1000` (or lower the score), or use cgroup limits. The victim shows exit status 137 (128 + 9, SIGKILL).
- 21 RSS counts a shared page fully in every process that maps it. Use PSS (private + shared / sharers), whose sum is accurate, or USS for unique memory.
- 22 Huge pages map 2 MiB (or 1 GiB) with one entry, cutting page-table size and TLB misses. THP advantage: automatic use, speedups for big-memory programs. Disadvantage: needs contiguous memory, can waste memory and cause latency spikes from compaction.

14.5 Glossary

Term	Meaning
Anonymous memory	Memory with no backing file: heap, stack, <code>MAP_ANONYMOUS</code> mappings.
Belady's anomaly	Getting more page faults with more frames (occurs with FIFO).
Buddy system	Allocator that splits and merges power-of-two blocks of pages.
Copy-on-write (COW)	Sharing pages read-only until a write forces a private copy.
Demand paging	Allocating and loading pages only when first accessed.
Direct reclaim	A process reclaiming memory itself because free memory is critically low.
Dirty page	A cached page modified in memory but not yet written to disk.
Huge page	A 2 MiB or 1 GiB page, mapped with one entry.
kswapd	Kernel thread that reclaims memory in the background.
LRU	Least Recently Used: evict what was used longest ago.
Major / minor fault	Page fault that does / does not need disk I/O.
MMU	Hardware that translates virtual to physical addresses.
NUMA	Non-uniform memory access: memory closer to some CPUs than others.
OOM killer	Kernel mechanism that kills a process when memory cannot be reclaimed.
Overcommit	Promising more memory than physically exists.
Page / page frame	A fixed-size (4 KiB) unit of virtual / physical memory.
Page cache	RAM used to cache file contents.
Page table	Structure mapping virtual pages to physical frames.
PSS / RSS / USS	Proportional / resident / unique memory of a process.
Slab allocator	Allocator for small, fixed-size kernel objects (SLUB in Linux).
Swap	Disk space holding anonymous pages evicted from RAM.
Thrashing	Constant paging because the working set exceeds RAM.

Term	Meaning
TLB	Cache of recent address translations inside the CPU.
VMA	Virtual memory area: a contiguous range of addresses with uniform properties.
Watermark	Free-memory threshold (min, low, high) that triggers reclaim.
Working set	The pages a program actively uses over a period of time.
Zone	A region of physical memory with common usability constraints (DMA, DMA32, Normal).

14.6 References and further reading

- **Linux manual pages:** `mmap(2)`, `brk(2)`, `mprotect(2)`, `madvise(2)`, `mlock(2)`, `fork(2)`, `proc(5)`, `free(1)`, `vmstat(8)`, `malloc(3)`, `mallopt(3)`, `numa(7)`, `cgroups(7)`. Read with `man 2 mmap`, `man 5 proc`, and so on.
- Linux kernel documentation: `Documentation/admin-guide/mm/` and `Documentation/mm/` in the kernel source, also online at kernel.org.
- Remzi and Andrea Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Free online. Chapters on address spaces, paging, TLBs, multi-level page tables, swapping, and replacement policies.
- Silberschatz, Galvin, Gagne. *Operating System Concepts*. Chapters on main memory, virtual memory, and page replacement.
- Michael Kerrisk. *The Linux Programming Interface*. Chapters on memory allocation, virtual memory operations, and memory mappings.
- Robert Love. *Linux Kernel Development*. Chapters on memory management and the page cache.
- Mel Gorman. *Understanding the Linux Virtual Memory Manager*. In-depth (older kernel), still very instructive.
- Kernel source (kernel.org): files listed in Section 14.2.

NOTE — LAST WORD

This guide was prepared as a study aid. Kernel details evolve with each release; when in doubt, consult the manual pages and source for the version you are running. **Generated for CMSC 125 ICS UPLB · Reviewed by JACH**